Graph Algorithms

Program Execution Report

CSC 506 — Design and Analysis of Algorithms | Module 7

1. Graph Representations: Vertex and Edge Storage

Both **graph_adj_list.py** and **graph_adj_matrix.py** correctly store and retrieve vertices and edges. The adjacency list stores neighbors as a dict mapping each label to a list of (neighbor, weight) tuples. The adjacency matrix stores a 2D weight grid where 0 = no edge.

Adjacency List output

```
$ python graph_adj_list.py
Graph after adding 5 vertices and 6 edges:
  A: B(1) -> C(1)
  B: A(1) -> D(1)
  C: A(1) -> D(1) -> E(1)
  D: B(1) -> C(1) -> E(1)
  E: C(1) -> D(1)

Neighbors of C: [('A', 1), ('D', 1), ('E', 1)]
has_edge(A, B): True
has_edge(A, E): False

Removing vertex B...

Graph after removing B:
  A: C(1)
  C: A(1) -> D(1) -> E(1)
  D: C(1) -> E(1)
  E: C(1) -> D(1)

--- Directed graph demo ---
Directed graph:
  A: B(1) -> C(1)
  B: D(1)
  C: D(1) -> E(1)
  D: E(1)
  E: None

has_edge(A, B): True
has_edge(B, A): False  (directed — no back-edge)
```

Adjacency Matrix output

```
$ python graph_adj_matrix.py
Graph after adding 5 vertices and 6 edges:
    A   B   C   D   E
   ------------------
 A |  0   1   1   0   0
 B |  1   0   0   1   0
 C |  1   0   0   1   1
```

```
 D |  0   1   1   0   1
 E |  0   0   1   1   0

Neighbors of C: ['A', 'D', 'E']
has_edge(A, B): True
has_edge(A, E): False

Removing vertex B...

Graph after removing B:
    A   C   D   E
   --------------
 A |  0   1   0   0
 C |  1   0   1   1
 D |  0   1   0   1
 E |  0   1   1   0
```

Both representations correctly enforce directed/undirected symmetry and return False for non-existent edges. Vertex removal propagates through all incident edges in both structures.

---

2. Traversal Algorithms: DFS and BFS

The traversal functions in **graph_traversal.py** are representation-agnostic. Both DFS (iterative stack) and BFS (queue) visit all 6 reachable vertices in identical order across both representations. The visual traversals display ASCII trees showing the discovered-edge tree structure.

```
$ python graph_traversal.py
============================================================
Directed graph — adjacency matrix:
    A   B   C   D   E   F
   ----------------------
 A |  0   1   1   0   0   0
 B |  0   0   0   1   1   0
 C |  0   0   0   1   0   1
 D |  0   0   0   0   1   0
 E |  0   0   0   0   0   1
 F |  0   0   0   0   0   0

Directed graph — adjacency list:
  A: B(1) -> C(1)
  B: D(1) -> E(1)
  C: D(1) -> F(1)
  D: E(1)
  E: F(1)
  F: None

============================================================
DFS from 'A' — adjacency MATRIX:
  Step 1: Visit A
  Step 2: Visit B
  Step 3: Visit D
  Step 4: Visit E
  Step 5: Visit F
  Step 6: Visit C

DFS from 'A' — adjacency LIST:
```

```
  Step 1: Visit A
  Step 2: Visit B
  Step 3: Visit D
  Step 4: Visit E
  Step 5: Visit F
  Step 6: Visit C

============================================================
BFS from 'A' — adjacency MATRIX:
  Step 1: Visit A
  Step 2: Visit B
  Step 3: Visit C
  Step 4: Visit D
  Step 5: Visit E
  Step 6: Visit F

BFS from 'A' — adjacency LIST:
  Step 1: Visit A
  Step 2: Visit B
  Step 3: Visit C
  Step 4: Visit D
  Step 5: Visit E
  Step 6: Visit F

[OK] Both representations produce identical traversal orders.

============================================================
VISUAL BFS
============================================================
BFS from 'A'
------------------------------------------------------
  Step 1  | Queue: [A]               | Visit: A  | Discovered: A
  Step 2  | Queue: [B, C]            | Visit: B  | Edge: B→D, B→E
  Step 3  | Queue: [C, D, E]         | Visit: C  | Edge: C→F
  Step 4  | Queue: [D, E, F]         | Visit: D  | Visit: D (no new edges)
  Step 5  | Queue: [E, F]            | Visit: E  | Visit: E (no new edges)
  Step 6  | Queue: [F]               | Visit: F  | Visit: F (no new edges)

  Traversal tree (BFS):

    A
    ■■■ B
    ■   ■■■ D
    ■   ■■■ E
    ■■■ C
        ■■■ F

============================================================
VISUAL DFS
============================================================
DFS from 'A'
--------------------------------------------
  [depth 0] Visit A
    [depth 1] Visit B
      [depth 2] Visit D
        [depth 3] Visit E
          [depth 4] Visit F
        [depth 3] Backtrack to E
      [depth 2] Backtrack to D
```

```
    [depth 1] Backtrack to B
  [depth 0] Backtrack to A
    [depth 1] Visit C
  [depth 0] Backtrack to A

  Traversal tree (DFS):

    A
    ■■■ B
    ■   ■■■ D
    ■       ■■■ E
    ■           ■■■ F
    ■■■ C

============================================================
Side-by-side traversal comparison
============================================================
Step  DFS (matrix)DFS (list)  BFS (matrix)BFS (list)
------------------------------------------------------
1     A           A           A           A
2     B           B           B           B
3     D           D           C           C
4     E           E           D           D
5     F           F           E           E
6     C           C           F           F
============================================================
```

---

3. Shortest Path: Dijkstra's Algorithm

The Dijkstra implementation in **shortest_path.py** uses a min-heap priority queue and prints every edge relaxation. Tested on a 7-vertex directed weighted graph, both representations produce identical distance tables and optimal paths.

```
$ python shortest_path.py
============================================================
DIJKSTRA — Adjacency List representation
============================================================

Relaxation trace (source = A):
  Relaxing edge A→B  old_dist=∞  new_dist=4
  Relaxing edge A→C  old_dist=∞  new_dist=2
  Relaxing edge C→B  old_dist=4  new_dist=3
  Relaxing edge C→D  old_dist=∞  new_dist=10
  Relaxing edge C→F  old_dist=∞  new_dist=12
  Relaxing edge B→D  old_dist=10  new_dist=8
  Relaxing edge B→E  old_dist=∞  new_dist=6
  Relaxing edge E→D  old_dist=8  new_dist=7
  Relaxing edge E→G  old_dist=∞  new_dist=12
  Relaxing edge D→G  old_dist=12  new_dist=9
  Relaxing edge F→G  old_dist=9  new_dist=15

Distance table (source = A):

 Destination    Distance  Path
-----------------------------------------------------
           B           3  A → C → B
           C           2  A → C
           D           7  A → C → B → E → D
```

```
           E           6  A → C → B → E
           F          12  A → C → F
           G           9  A → C → B → E → D → G

======================================================
DIJKSTRA — Adjacency Matrix representation
======================================================

Relaxation trace (source = A):
  Relaxing edge A→B  old_dist=∞  new_dist=4
  Relaxing edge A→C  old_dist=∞  new_dist=2
  Relaxing edge C→B  old_dist=4  new_dist=3
  Relaxing edge C→D  old_dist=∞  new_dist=10
  Relaxing edge C→F  old_dist=∞  new_dist=12
  Relaxing edge B→D  old_dist=10  new_dist=8
  Relaxing edge B→E  old_dist=∞  new_dist=6
  Relaxing edge E→D  old_dist=8  new_dist=7
  Relaxing edge E→G  old_dist=∞  new_dist=12
  Relaxing edge D→G  old_dist=12  new_dist=9
  Relaxing edge F→G  old_dist=9  new_dist=15

Distance table (source = A):

 Destination    Distance  Path
------------------------------------------------------
           B           3  A → C → B
           C           2  A → C
           D           7  A → C → B → E → D
           E           6  A → C → B → E
           F          12  A → C → F
           G           9  A → C → B → E → D → G

======================================================
Assertion passed: both representations yield
identical distances and paths for all vertices.
======================================================
```

---

4. Performance Analysis: Benchmark Results

benchmark.py tested graph sizes N = 50 to 2,000 with sparse edge density (E = N·ln N). Timings are best-of-3 runs in milliseconds.

```
$ python benchmark.py
```

Build Graph (N vertices + N·ln N edges)

N	MatrixGraph (ms)	ListGraph (ms)	Ratio (List/Matrix)
50	0.0524	0.0375	0.71x
100	0.1474	0.0877	0.60x
500	1.9160	0.7308	0.38x
1000	6.2602	1.6099	0.26x
2000	24.2038	3.5801	0.15x

BFS Traversal (from vertex 0)

N	MatrixGraph (ms)	ListGraph (ms)	Ratio (List/Matrix)
50	0.0510	0.0172	0.34x
100	0.1463	0.0367	0.25x
500	2.7917	0.2707	0.10x
1000	11.2397	0.5585	0.05x
2000	44.6634	1.1896	0.03x

Dijkstra's Algorithm (single-source)

N	MatrixGraph (ms)	ListGraph (ms)	Ratio (List/Matrix)
50	0.0822	0.0334	0.41x
100	0.2256	0.0757	0.34x
500	3.2990	0.4714	0.14x
1000	12.4367	1.1037	0.09x
2000	47.7413	2.5534	0.05x

Edge Lookup (1,000 random has_edge calls)

N	MatrixGraph (ms)	ListGraph (ms)	Ratio (List/Matrix)
50	0.0813	0.2947	3.63x
100	0.0813	0.3175	3.90x
500	0.1024	0.4759	4.65x
1000	0.1101	0.4840	4.40x
2000	0.1166	0.5098	4.37x

Trade-off Summary

Operation	Winner	Complexity
Build	Adjacency List	$O(N+E)$ vs $O(N^2)$
BFS / DFS	Adjacency List	$O(V+E)$ vs $O(V^2)$
Dijkstra	Adjacency List	$O((V+E)\log V)$ vs $O(V^2 \log V)$
Edge Lookup	Adjacency Matrix	$O(1)$ vs $O(\text{degree})$
Space (sparse)	Adjacency List	$O(V+E)$ vs $O(V^2)$

Build / Traversal / Dijkstra: The adjacency list wins on sparse graphs because vertex insertion is O(1) and neighbor iteration is O(degree). The matrix pays O(V) per vertex scan regardless of edge count, making it $O(V^2)$ for traversals and $O(N^2)$ to build.

Edge Lookup: The matrix wins unconditionally — two dict lookups + one array access = O(1). The list must scan the neighbor list linearly: O(degree).

Space: The matrix allocates V×V cells always ($O(V^2)$). At N=2,000 that is 4M integers. The list uses O(V+E) — at E≈N·ln N, roughly O(N).

---

5. Test Suite: 10/10 Tests Pass

test_graphs.py runs a comprehensive suite on an 8-vertex, 12-edge undirected weighted graph with known shortest path A→H = cost 9 via A→B(3)→E(4)→H(2).

```
$ python test_graphs.py
======================================================================
Graph Test Suite
======================================================================
Test  1  PASS  add_vertex / has_edge round-trip on both representations
Test  2  PASS  remove_vertex cleans up all incident edges
Test  3  PASS  DFS visits all reachable vertices, correct order
Test  4  PASS  BFS visits all reachable vertices, correct order
Test  5  PASS  DFS and BFS produce same visited set (not same order)
Test  6  PASS  Dijkstra finds the known shortest path A→H = cost 9
Test  7  PASS  Dijkstra returns inf for unreachable vertex in directed graph
Test  8  PASS  Both representations return identical Dijkstra results
Test  9  PASS  display() produces output with correct vertex count
Test 10  PASS  Performance: matrix and list both complete 1000-vertex BFS in under 5 seconds
======================================================================
10/10 tests passed
```

Test	Description	Result
1	add_vertex / has_edge round-trip on both representations	PASS
2	remove_vertex cleans up all incident edges	PASS
3	DFS visits all reachable vertices, correct order	PASS
4	BFS visits all reachable vertices, correct order	PASS
5	DFS and BFS produce same visited set (not same order)	PASS
6	Dijkstra finds the known shortest path A→H = cost 9	PASS
7	Dijkstra returns inf for unreachable vertex in directed graph	PASS
8	Both representations return identical Dijkstra results	PASS
9	display() produces output with correct vertex count	PASS
10	Matrix and list both complete 1,000-vertex BFS in under 5 seconds	PASS

---

6. Practical Application: USPS Denver Delivery Route

usps_denver_dataset.py models the Capitol Hill, Denver (ZIP 80203) USPS letter-carrier network — 17 nodes (post office + 16 street intersections), 35 directed edges respecting real one-way streets (Colfax Ave eastbound; Logan/Pearl southbound; Pennsylvania/Clarkson northbound). Dijkstra computes the shortest walking distance in feet from the Post Office to every intersection.

Key Delivery Distances (both representations agree exactly)

Destination	Distance (ft)	Optimal Route
12_LOG	400	PO → 12_PENN → 12_LOG
COL_PENN	450	PO → 12_PENN → COL_PENN
14_PENN	800	PO → 12_PENN → COL_PENN → 14_PENN
COL_CLK	1,050	PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK
15_PENN	1,150	PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN
14_CLK	1,400	PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK
15_LOG	1,460	PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_LOG
15_CLK	1,750	PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 15_CLK

BFS Delivery Layers (fewest street segments from Post Office)

Layer	Intersections
0	PO
1	12_PENN
2	12_LOG, 12_PEARL, COL_PENN
3	12_CLK, COL_PEARL, 14_PENN
4	COL_CLK, 14_LOG, 14_PEARL, 15_PENN
5	14_CLK, COL_LOG, 15_LOG, 15_PEARL
6	15_CLK

All 17/17 intersections reachable from PO. **8/8 known-route assertions passed for both adjacency list and adjacency matrix.**

```
$ python usps_denver_dataset.py
======================================================================
Denver, CO — Capitol Hill USPS Delivery Route
======================================================================
  City     : Denver, Colorado (ZIP 80203)
  Area     : Capitol Hill — E 12th Ave to E 15th Ave,
             Logan St to Clarkson St
  Vertices : 17  (post office + 16 intersections)
  Edges    : 35  (directed; one-way streets explicit)
  Weights  : street distance in feet
  Source   : PO (Capitol Hill Station Post Office)

======================================================================
DIJKSTRA DELIVERY TABLE — Adjacency List
Source: Capitol Hill Station Post Office (PO)
======================================================================
  Destination      Distance (ft)  Route
  --------------  --------------  ----------------------------------
  12_LOG                    400  PO → 12_PENN → 12_LOG
  12_PENN                   100  PO → 12_PENN
  12_PEARL                  400  PO → 12_PENN → 12_PEARL
  12_CLK                    700  PO → 12_PENN → 12_PEARL → 12_CLK
  COL_LOG                 1,460  PO → 12_PENN → COL_PENN → 14_PENN → 14_LOG → COL_LOG
```

```
  COL_PENN                   450  PO → 12_PENN → COL_PENN
  COL_PEARL                  780  PO → 12_PENN → COL_PENN → COL_PEARL
  COL_CLK                  1,050  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK
  14_LOG                   1,110  PO → 12_PENN → COL_PENN → 14_PENN → 14_LOG
  14_PENN                    800  PO → 12_PENN → COL_PENN → 14_PENN
  14_PEARL                 1,110  PO → 12_PENN → COL_PENN → 14_PENN → 14_PEARL
  14_CLK                   1,400  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK
  15_LOG                   1,460  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_LOG
  15_PENN                  1,150  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN
  15_PEARL                 1,460  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_PEARL
  15_CLK                   1,750  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 15_CLK

======================================================================
DIJKSTRA DELIVERY TABLE — Adjacency Matrix
Source: Capitol Hill Station Post Office (PO)
======================================================================
  Destination      Distance (ft)  Route
  -------------  --------------  ----------------------------------
  12_LOG                   400  PO → 12_PENN → 12_LOG
  12_PENN                  100  PO → 12_PENN
  12_PEARL                 400  PO → 12_PENN → 12_PEARL
  12_CLK                   700  PO → 12_PENN → 12_PEARL → 12_CLK
  COL_LOG                1,460  PO → 12_PENN → COL_PENN → 14_PENN → 14_LOG → COL_LOG
  COL_PENN                 450  PO → 12_PENN → COL_PENN
  COL_PEARL                780  PO → 12_PENN → COL_PENN → COL_PEARL
  COL_CLK                1,050  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK
  14_LOG                 1,110  PO → 12_PENN → COL_PENN → 14_PENN → 14_LOG
  14_PENN                  800  PO → 12_PENN → COL_PENN → 14_PENN
  14_PEARL               1,110  PO → 12_PENN → COL_PENN → 14_PENN → 14_PEARL
  14_CLK                 1,400  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK
  15_LOG                 1,460  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_LOG
  15_PENN                1,150  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN
  15_PEARL               1,460  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_PEARL
  15_CLK                 1,750  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 15_CLK

======================================================================
TRAVERSAL COVERAGE — Adjacency List
======================================================================
  BFS from PO  (17/17 intersections):
    PO → 12_PENN → 12_LOG → 12_PEARL → COL_PENN → 12_CLK → COL_PEARL → 14_PENN → COL_CLK → 14_LOG → 14_PEARL → 1

  DFS from PO  (17/17 intersections):
    PO → 12_PENN → 12_LOG → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 14_PEARL → 14_PENN → 14_LOG → COL_LOG → COL_P

  All 17 intersections reachable from PO.

  BFS delivery layers (fewest turns from PO):
    Layer 0: PO
    Layer 1: 12_PENN
    Layer 2: 12_LOG, 12_PEARL, COL_PENN
    Layer 3: 12_CLK, COL_PEARL, 14_PENN
    Layer 4: COL_CLK, 14_LOG, 14_PEARL, 15_PENN
    Layer 5: 14_CLK, COL_LOG, 15_LOG, 15_PEARL
    Layer 6: 15_CLK

======================================================================
ASSERTION CHECKS [adj-list]
======================================================================
  PASS  PO → 12_LOG           400 ft  PO → 12_PENN → 12_LOG
```

```
  PASS  PO → COL_PENN         450 ft  PO → 12_PENN → COL_PENN
  PASS  PO → 14_PENN          800 ft  PO → 12_PENN → COL_PENN → 14_PENN
  PASS  PO → 15_PENN        1,150 ft  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN
  PASS  PO → COL_CLK        1,050 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK
  PASS  PO → 14_CLK         1,400 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK
  PASS  PO → 15_LOG         1,460 ft  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_LOG
  PASS  PO → 15_CLK         1,750 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 15_CLK

  8/8 assertions passed [adj-list]

======================================================================
ASSERTION CHECKS [adj-matrix]
======================================================================
  PASS  PO → 12_LOG           400 ft  PO → 12_PENN → 12_LOG
  PASS  PO → COL_PENN         450 ft  PO → 12_PENN → COL_PENN
  PASS  PO → 14_PENN          800 ft  PO → 12_PENN → COL_PENN → 14_PENN
  PASS  PO → 15_PENN        1,150 ft  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN
  PASS  PO → COL_CLK        1,050 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK
  PASS  PO → 14_CLK         1,400 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK
  PASS  PO → 15_LOG         1,460 ft  PO → 12_PENN → COL_PENN → 14_PENN → 15_PENN → 15_LOG
  PASS  PO → 15_CLK         1,750 ft  PO → 12_PENN → 12_PEARL → 12_CLK → COL_CLK → 14_CLK → 15_CLK

  8/8 assertions passed [adj-matrix]

======================================================================
All assertions passed for both graph representations.
======================================================================
```

Summary

Criterion	Result
Graph representations store vertices and edges correctly	Verified — both handle directed/undirected, weighted/unweighted
Manipulation (add/remove) works correctly	Verified — vertex removal cleans all incident edges in both
Traversal visits all reachable vertices in correct order	Verified — DFS and BFS identical across both representations
Shortest path finds optimal routes	Verified — Dijkstra correct with full relaxation trace
Performance analysis compares trade-offs	Verified — list dominates sparse traversal; matrix wins O(1) lookup
Practical application demonstrated	Verified — USPS Denver route, 8/8 real-world assertions passed